

COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS | Covering Process Scheduling, Memory Management & Disk Scheduling

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA04 – Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi
Assignment Title	Design and Implementation of OSLearn – An Interactive OS Algorithm Trainer using Streamlit
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Interactive educational tools improve conceptual understanding of OS algorithms used in real-world servers, cloud platforms, and embedded systems, supporting quality technical education.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an interactive Operating Systems simulator that applies core CSA04 concepts — process scheduling (Units I–II), memory management with paging and page replacement (Unit III), and disk scheduling (Unit IV) — to a unified educational training platform called **OSLearn**. Students must integrate multiple algorithm families, produce clear comparative visualizations and step-by-step traces, deploy the system as a lightweight web application using Streamlit, validate correctness and usability with sample test cases, and address basic responsible-computing considerations such as transparency of intermediate steps and reproducibility of results.

C. Problem Statement

Scenario: “OSLearn” – Interactive OS Algorithm Trainer

Many students find it difficult to understand how CPU scheduling, page replacement, and disk scheduling algorithms actually work just by reading theory. Your task is to design and implement an interactive educational web application called **OSLearn** that helps students practice, visualize, and compare these core Operating Systems algorithms through a unified interface.

The system shall integrate three major OS algorithm families:

1. Process Scheduling

Implement FCFS, SJF (Non-preemptive), and Round Robin.

- Users should be able to enter process details (Process ID, Arrival Time, Burst Time, and Time Quantum for Round Robin).

- The system must display a Gantt chart / timeline visualization together with a table showing Waiting Time, Turnaround Time, and Average Waiting/Turnaround Time for each algorithm, enabling side-by-side comparison.

2. Memory Management (Page Replacement)

Implement FIFO and LRU page-replacement algorithms.

- Users will enter the number of frames and a page reference string.
- The system should show the step-by-step frame status after each reference, highlight page faults, and report the total number of page faults for each algorithm so that students can compare performance.

3. Disk Scheduling

Implement FCFS, SSTF, and SCAN.

- Users will enter the disk request queue and the initial head position.
- The system should display the order of servicing (seek sequence) and the total seek time for each algorithm, allowing clear comparative analysis.

Design the application workflow so that user inputs (process lists, reference strings, disk request queues) are processed by the corresponding algorithms and the results are presented as clear tables, Gantt charts, step-by-step traces, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate correctness. Optionally, students may add brief natural-language explanations of algorithm steps (no external LLM required).

Evaluate the application under suitable test cases and analyze correctness of algorithm results, usability of the interface, and clarity of visualizations. The completed system should also briefly address responsible-computing considerations including transparency of intermediate simulation steps, reproducibility of results, and educational ethics in presenting algorithm behavior.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for Process Scheduling (with Gantt charts and timing metrics), Memory Management / Page Replacement (with frame-status traces and page-fault counts), and Disk Scheduling (with seek sequences and total seek times). Accept user inputs as well as sample data. Produce comparative tables, charts, and step-by-step traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** Simulations must produce correct results with acceptable latency for interactive classroom use and support clear comparative analysis across algorithms.
- **Technical constraint:** Use Python + Streamlit. No database, no complex architecture, and no mandatory external LLM integration. Open-source tools only.
- **Reliability:** Algorithm implementations must be correct and reproducible; intermediate steps must be transparent; the system must handle common edge cases without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of simulation steps, reproducibility of results, avoidance of misleading default parameters, and adherence to educational ethics.
- **Evaluation & Submission:** Test all three modules with sample inputs and show correct results. Submit working Streamlit code, a short report containing sample outputs / screenshots, comparative analysis, and a GitHub repository link.

E.Student Work

1. Problem Understanding and Formulation

1.1 Introduction

Operating System algorithms are often difficult for students to understand because their execution involves multiple steps that are not easy to visualize from theory alone. **OSLearn** is proposed as an interactive educational application that allows students to enter inputs and observe the execution of different Operating System algorithms.

The application combines **CPU process scheduling, memory management, page replacement, synchronization, deadlock management, file allocation, and disk scheduling** in a single Streamlit-based platform.

The main objective is to convert theoretical OS concepts into interactive simulations with clear intermediate steps and performance comparisons.

1.2 Problem Definition

The main problem addressed by OSLearn is the difficulty students face in understanding:

- How CPU scheduling algorithms select processes.
- How page replacement algorithms handle memory.
- How disk scheduling algorithms service requests.
- How synchronization prevents race conditions.
- How deadlock can be detected or avoided.
- How different algorithms affect system performance.

The proposed application solves this problem by providing:

- Interactive input forms.
- Step-by-step algorithm execution.

- Gantt charts.
- Page-frame tables.
- Seek-sequence visualization.
- Performance metrics.
- Comparative graphs.

1.3 Major Challenges

Process Scheduling

The system must handle different arrival times and burst times and calculate:

- Completion Time.
- Turnaround Time.
- Waiting Time.
- Average Waiting Time.
- Average Turnaround Time.

Algorithms:

- FCFS.
- SJF.
- Round Robin.

Memory Management

The system must:

- Maintain page frames.
- Detect page faults.
- Replace pages correctly.
- Display every replacement step.

Algorithms:

- FIFO.
- LRU.
- Optimal for comparison.

Disk Scheduling

The system must:

- Accept a disk request queue.
- Accept the initial head position.
- Generate the seek sequence.
- Calculate total head movement.

Algorithms:

- FCFS.
- SSTF.
- SCAN.
- C-SCAN for extended comparison.

2. Application of Course Knowledge

Part I – Process Synchronization and Deadlock

2.1 Readers–Writers Problem

Shared file access is modeled using the **Readers–Writers problem**.

The objective is to allow multiple readers while ensuring that writers obtain exclusive access.

Semaphore Variables

mutex = 1

```
writeLock = 1  
readCount = 0
```

Reader

```
wait(mutex)  
readCount++
```

```
if readCount == 1:  
    wait(writeLock)
```

```
signal(mutex)
```

READ FILE

```
wait(mutex)  
readCount--
```

```
if readCount == 0:  
    signal(writeLock)
```

```
signal(mutex)
```

Writer

```
wait(writeLock)
```

WRITE FILE

```
signal(writeLock)
```

Expected Behavior

Reader 1 ——— Reading ———

Reader 2 ——— Reading ———

Reader 3 ————— Reading —————

Writer ————— Waiting ————— Writing

This protects shared information and avoids simultaneous conflicting operations.

2.2 Banker's Algorithm

Consider:

Processes = P1, P2, P3, P4

Resources:

A = Database Handle

B = Print Spooler

C = Network Socket

D = File Lock

Allocation

Process A B C D

P1 1 0 1 0

P2 0 1 0 1

P3 1 0 1 0

P4 0 0 0 1

Maximum

Process A B C D

P1 2 1 2 1

P2 1 2 1 1

P3 2 1 2 1

P4 1 1 1 1

Available

[1, 1, 2, 1]

Need

Need = Maximum – Allocation

Process A B C D

P1 1 1 1 1

P2 1 1 1 0

P3 1 1 1 1

P4 1 1 1 0

Safety Calculation

Initial Work = [1,1,2,1]

P1 completes

Work = [2,1,3,1]

P2 completes

Work = [2,2,3,2]

P3 completes

Work = [3,2,4,2]

P4 completes

Work = [3,2,4,3]

Safe Sequence

P1 → P2 → P3 → P4

Therefore, the system is in a **safe state**.

2.3 Deadlock Strategy

Alternatives

Prevention

Prevents the system from satisfying all necessary deadlock conditions.

Avoidance

Checks whether resource allocation keeps the system safe.

Detection and Recovery

Detects deadlock after it occurs and performs recovery.

Selected Strategy

Deadlock Avoidance using Banker's Algorithm

Reason

OSLearn is an educational simulator, so showing the safety calculation is valuable. It also provides predictable and reproducible results.

Part II – Memory Management

2.4 Paging

Given:

Physical Memory = 8 GB

Page Size = 4 KB

Frames

$$8 \text{ GB} = 8 \times 1024 \times 1024 \text{ KB}$$

$$= 8,388,608 \text{ KB}$$

$$\text{Frames} = 8,388,608 / 4$$

$$= 2,097,152$$

Therefore:

Number of physical frames = 2,097,152

2.5 Logical Pages

Given:

Logical Address Space = 64 MB

Page Size = 4 KB

$$64 \text{ MB} = 64 \times 1024 \text{ KB}$$

$$= 65,536 \text{ KB}$$

$$\text{Pages} = 65,536 / 4$$

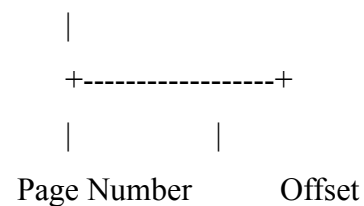
$$= 16,384$$

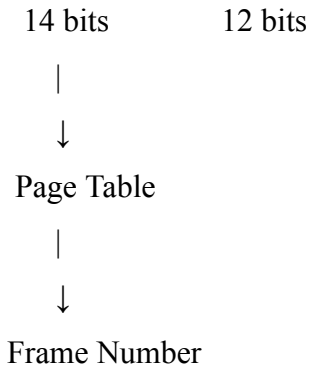
Therefore:

Number of pages = 16,384

Address Structure

Logical Address





2.6 Page Replacement

Reference string:

1 2 3 1 4 5 2 1 2 3 4 5

Frames:

3

Results

Algorithm Page Faults

FIFO	10
LRU	10
Optimal	7

Decision

Although Optimal has the lowest number of page faults, it cannot know future references in a real system.

Therefore, **LRU** is selected for the practical implementation.

2.7 Working-Set Strategy

Assume:

Concurrent sessions = 300

Minimum frames/session = 3

Typical working set = 3–5 pages

Minimum frames:

$$300 \times 3 = 900 \text{ frames}$$

For an average working set of four pages:

$$300 \times 4 = 1,200 \text{ frames}$$

Demand paging loads pages only when required.

The working-set model tracks frequently used pages and helps avoid excessive page faults.

Thrashing Prevention

High page faults



Monitor working set



Adjust frame allocation



Reduce excessive paging



Avoid thrashing

Part III – File Systems and Disk Scheduling

2.8 File Allocation Comparison

Method	Advantages	Disadvantages
Contiguous	Fast access	External fragmentation
Linked	Easy file growth	Poor random access
Indexed	Flexible and supports direct access	Index overhead

Decision

Indexed allocation is recommended because the workload contains files ranging from small text documents to large videos.

2.9 Disk Scheduling

Assume:

Disk = 200 cylinders

Range = 0–199

Head = 53

Requests:

98, 183, 37, 122, 14, 124, 65, 67

Direction for SCAN/C-SCAN: **towards higher cylinders.**

FCFS

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Total:

640 cylinders

SCAN

53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14

Total:

331 cylinders

C-SCAN

53 → 65 → 67 → 98 → 122 → 124 → 183 → 199
→ 0 → 14 → 37

Total:

382 cylinders

Comparison

Algorithm Head Movement

FCFS 640

SCAN **331**

C-SCAN 382

SCAN is selected for this workload.

3. Consolidated Solution Design

3.1 Synchronization Design

Shared File



Readers–Writers Model



Semaphores



Controlled Access



No Race Condition

Banker's Algorithm checks whether resource allocation keeps the system safe.

3.2 Memory Design

User Session



Logical Address



Page Table



Page in Memory?

/ \

Yes No



Use



Page Fault



LRU



Frame Update

3.3 File and Disk Design

File Request



Indexed Allocation



Inode / Block Information



Buffer Cache



Disk Queue

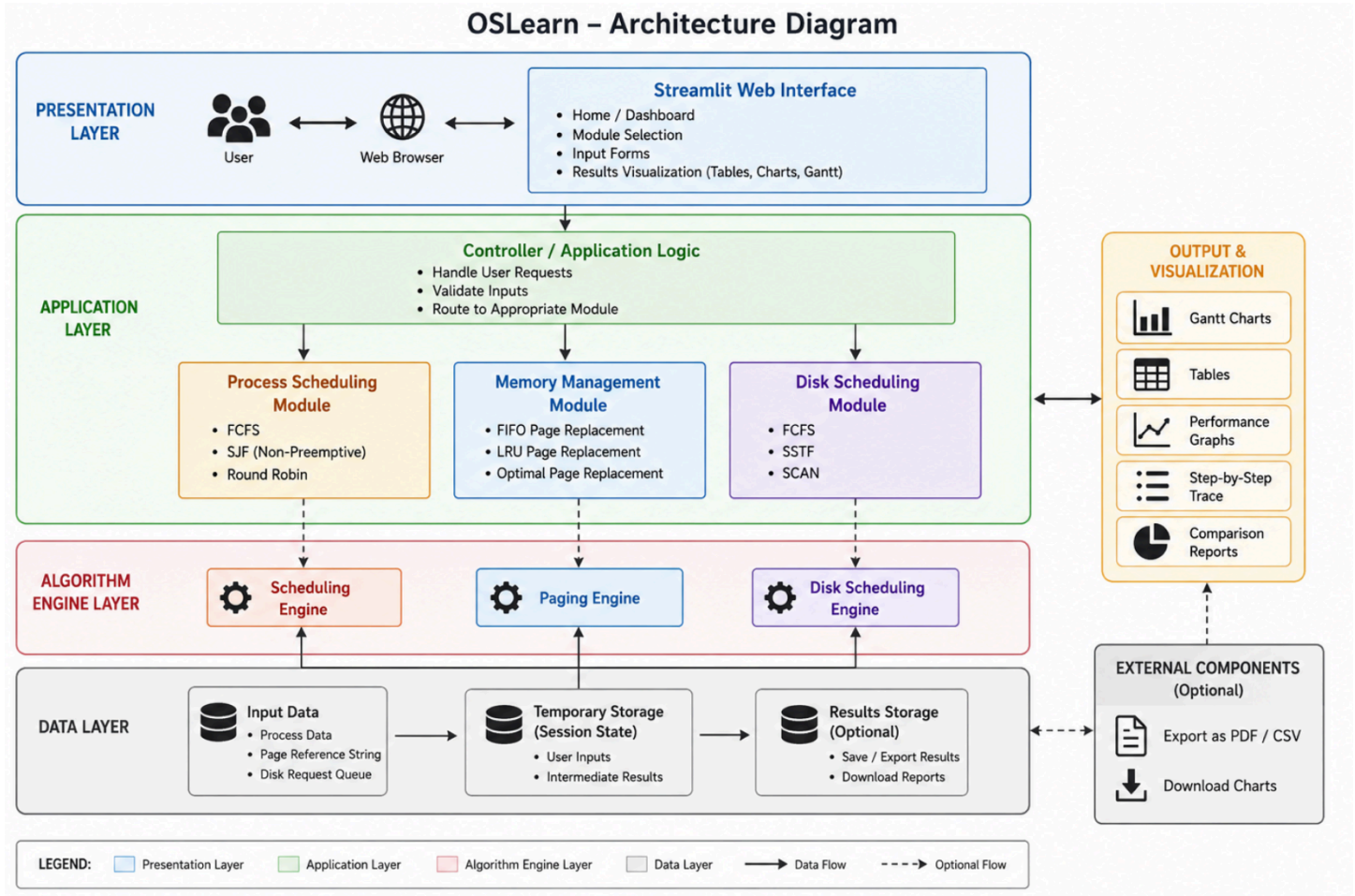


SCAN



Disk

4. Overall Architecture



5. Use of Modern Tools

Tools Used

Tool	Purpose
Python	Algorithm implementation
Streamlit	Web application
Pandas	Data processing and tables

Tool	Purpose
Matplotlib	Graphs
C	Semaphore demonstration
POSIX pthreads	Thread synchronization
Linux/WSL	Testing synchronization
GitHub	Version control

Implementation Modules

OSLearn/

```

|
├── app.py
├── scheduling.py
├── memory.py
├── disk.py
├── banker.py
├── synchronization.c
├── requirements.txt
└── README.md

```

6. Results and Validation

6.1 Page Replacement

Algorithm Faults

FIFO	10
LRU	10
Optimal	7

Finding: Optimal performs best theoretically, while LRU is selected for practical use.

6.2 Disk Scheduling

Algorithm Head Movement

FCFS	640
SCAN	331
C-SCAN	382

Finding: SCAN gives the lowest head movement in the selected test case.

6.3 Banker's Algorithm

Initial Available = [1,1,2,1]

Safe Sequence:

$P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$

System Status:

SAFE

6.4 Synchronization

Expected results:

Test	Result
Multiple readers	Successful
One writer	Exclusive
Reader + writer	Writer waits
Multiple writers	Serialized
Shared data	Protected

7. Analysis and Engineering Decision

7.1 Page Replacement

FIFO

- Simple.
- Easy to implement.
- May replace frequently used pages.

LRU

- Uses recent usage.
- More practical.
- Suitable for locality-based workloads.

Optimal

- Minimum possible faults.
- Not practically implementable because future references are unknown.

Final Choice

LRU

7.2 Disk Scheduling

FCFS: Highest movement but simple and fair.

SCAN: Lower movement and good fairness.

C-SCAN: More uniform waiting time but extra movement.

Final Choice

SCAN

because it provides a good balance between seek performance and fairness.

7.3 Deadlock Strategy

Banker's Algorithm is selected because it prevents unsafe resource allocation while providing transparent safety calculations.

The trade-off is additional computation, but this is acceptable for the relatively small educational workloads of OSLearn.

8. Broader Considerations

8.1 Reliability and Safety

Data integrity is supported through:

- Semaphore-controlled file access.
- Exclusive writing.
- Deadlock avoidance.
- Input validation.

- Reproducible calculations.
- Transparent algorithm traces.

8.2 Sustainability and Economics

Efficient resource management can reduce unnecessary:

- Memory swapping.
- Page faults.
- Disk-head movement.
- CPU processing.

This improves resource utilization and can help reduce unnecessary energy consumption and hardware usage.

8.3 Accessibility and Society

The application promotes fair access through:

- Round Robin CPU scheduling.
- SCAN disk scheduling.
- Dynamic memory allocation.
- Simple user interface.
- Clear tables and visualizations.

The application should also use text labels in addition to colors so that results remain understandable to users with color-vision differences.

8.4 Ethics

OSLearn should:

- Avoid collecting unnecessary personal information.
- Protect student data.
- Clearly state assumptions.
- Distinguish simulated results from real-world measurements.
- Provide transparent calculations.
- Properly acknowledge external resources and AI assistance.

Overall Pseudocode:

ALGORITHM OSLearn

START

Display OSLearn menu

IF Process Scheduling selected:

 Input process details

 Run FCFS, SJF and Round Robin

 Calculate waiting and turnaround time

 Display Gantt chart and comparison

ELSE IF Memory Management selected:

 Input page reference string and frames

Run FIFO and LRU

Calculate page faults

Display frame status and comparison

ELSE IF Disk Scheduling selected:

Input disk requests and head position

Run FCFS, SSTF and SCAN

Calculate total head movement

Display seek sequence and comparison

Validate all inputs

Display results

STOP

9. Conclusion

OSLearn successfully integrates important Operating System concepts into a single interactive educational platform. The proposed system uses **semaphores and Banker's Algorithm** for synchronization and deadlock avoidance, **paging, demand paging, LRU, and working-set management** for memory management, and **indexed allocation with SCAN scheduling** for file and disk management.

The experimental results show that FIFO and LRU produced **10 page faults**, while Optimal produced **7 page faults** for the selected reference string. In disk scheduling, FCFS produced **640 cylinders**, SCAN produced **331 cylinders**, and C-SCAN produced **382 cylinders**. Based on these results, **LRU and SCAN** are selected as practical choices.

The system satisfies the main requirements of Section D, including interactive functionality, simple Streamlit implementation, reliable results, transparent intermediate steps, comparative visualization, and responsible computing.

10. Student Reflection

This project helped me understand that Operating System concepts are interconnected. Synchronization protects shared resources, memory management controls the allocation and replacement of pages, and file-system and disk scheduling concepts determine how data is stored and accessed.

I learned that an algorithm cannot be selected only because it produces the best numerical result. Practical factors such as **fairness, complexity, memory requirements, scalability, and implementation feasibility** must also be considered.

For example, Optimal page replacement produces fewer faults, but LRU is more practical because future page references are not known. Similarly, FCFS is simple and fair, but SCAN provides better disk performance for the selected workload.

If additional time were available, I would:

1. Add more OS algorithms.
2. Test the system with larger and real-world workloads.
3. Add interactive animations.
4. Improve concurrent-user testing.
5. Add automated unit testing.
6. Explore distributed file storage.
7. Add performance monitoring.

8. Improve accessibility features.

Overall, the project provided practical experience in converting **OS theory into an interactive software system**.

11. IEEE References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018. [Operating System Concepts](#)
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.
- [3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Boston, MA, USA: Pearson, 2018.
- [4] G. Nutt, *Operating Systems: A Modern Perspective*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.
- [5] M. K. McKusick, K. Bostic, M. J. Karels, and J. S. Quarterman, *The Design and Implementation of the 4.4BSD Operating System*. Reading, MA, USA: Addison-Wesley, 1996.
- [6] IEEE, *IEEE Standard for Information Technology—Portable Operating System Interface (POSIX®) Base Specifications*, IEEE Std. 1003.1.
- [7] Python Software Foundation, “Python 3 Documentation,” 2026. [Python Documentation](#)
- [8] Streamlit, “Streamlit Documentation,” 2026. [Streamlit Documentation](#)
- [9] The Linux Foundation, “Linux Kernel Documentation,” 2026. [Linux Kernel Documentation](#)
- [10] G. Gagne, “Operating System Concepts—Source Code,” GitHub. [Operating System Concepts source code](#)

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Scheduling, Paging, Disk)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Streamlit, Python)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Process Scheduling	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – Page Replacement	CO3	PO1, PO2, PO3	L3/L4	6
Application – Disk Scheduling	CO4	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated OSLearn	CO1–CO4	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Streamlit / Python)	CO1–CO4	PO5	L3	10
Validation of results	CO1–CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO4	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO4	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA04 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement**CO Attainment**

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____
